

Dynamic Programming: 1D DP

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

June 29, 2026

Contents

1	Dynamic Programming: 1D DP	1
1.1	From Recursion to Tabulation: The Four Stages	1
1.2	Fibonacci Number	3
1.3	Climbing Stairs	5
1.4	Frog Jump	8
1.5	Frog Jump with K Distances	11
1.6	House Robber	14
1.7	House Robber II (Houses in a Circle)	17

Chapter 1

Dynamic Programming: 1D DP

Dynamic programming is not a different way of thinking from recursion – it is recursion, plus a memory. Find the correct recursive recurrence first, exactly as in the previous chapter; only afterward ask whether that recurrence revisits the same sub-problem more than once, and if it does, give it a memory.

1.1 From Recursion to Tabulation: The Four Stages

The Recursion chapter ended on its Word Break section, which exposed a problem whose correct recursive solution recomputes the same sub-problems exponentially many times. This chapter, and every dynamic programming chapter that follows, is about systematically fixing exactly that weakness. Every problem here is solved in up to four progressively refined stages, and the chapter’s single most important habit is deriving them *in this order*, never skipping straight to the last one.

The Four Stages, Illustrated on Fibonacci

Consider computing the n -th Fibonacci number, $F(n) = F(n-1) + F(n-2)$, with $F(0) = 0$, $F(1) = 1$.

1. **Recursion (brute force).** Translate the recurrence directly into a function: $\text{FIB}(n)$ returns n if $n \leq 1$, else $\text{FIB}(n-1) + \text{FIB}(n-2)$. This is always correct by definition, but $\text{FIB}(n-2)$ is computed once directly here and *again* inside the recursive expansion of $\text{FIB}(n-1)$ – and this duplication compounds at every level, giving $O(2^n)$ time.
2. **Memoization (top-down DP).** Add an array (or hash map) *memo*, indexed by the same parameter(s) the recursive function takes. Before computing $\text{FIB}(n)$ ’s result, check whether *memo*[n] is already filled; if so, return it directly. Otherwise compute it exactly as in the brute-force recursion, but **store** the result in *memo*[n] before returning. This is the *exact same* recursive function, with two added lines – yet it drops the time complexity to $O(n)$, since each distinct sub-problem $\text{FIB}(k)$ is now genuinely computed only once.
3. **Tabulation (bottom-up DP).** Rewrite the same recurrence as an explicit loop filling an array *dp* from the base cases *upward* to n , instead of recursing downward from n : $dp[0] = 0$, $dp[1] = 1$, and for i from 2 to n : $dp[i] = dp[i-1] + dp[i-2]$. This computes exactly the same values as memoization, in the same $O(n)$ time, but with no recursion or call-stack overhead at all.
4. **Space Optimization.** Observe that computing $dp[i]$ only ever needs $dp[i-1]$ and $dp[i-2]$ – once $dp[i]$ is computed, every earlier entry is permanently unneeded. Replace the full array with two scalar variables, updated in a rolling fashion, cutting space from $O(n)$ to $O(1)$.

Why Derive All Four, Rather Than Jumping to the Last

Tabulation and space optimization are mechanical rewrites of memoization, which is itself a two-line addition to brute-force recursion – but each rewrite is only safe once the *previous* stage is verified correct. Attempting to write a tabulation loop directly, without first confirming the recursive recurrence by hand on a small example, is the single most common source of off-by-one and base-case bugs in dynamic programming. This chapter therefore presents every problem in exactly this order: state the recursive recurrence and verify it on a small case, add memoization, convert to tabulation, then space-optimize – and the *only* thing that changes from problem to problem is what the recurrence itself looks like.

How to Recognise a 1D DP Problem and Find Its Recurrence

- **“Number of ways to reach step n ,” or “minimum cost/jumps to reach position n ”** – the answer at position n is built from the answers at a small, fixed set of earlier positions (e.g. $n-1$ and $n-2$ for single or double steps), combined by addition (counting ways) or by minimization (cost).
- **“Maximum sum, no two adjacent chosen” (House Robber and its variants)** – at each position, the recurrence is a *choice*: include this position’s value plus the best answer from two positions back, or exclude it and take the best answer from one position back, whichever is larger.

Once the recurrence is identified, the parameter(s) it depends on become the memoization array’s index (or indices) – this is true for every DP problem in this book, not just the ones in this chapter.

Each section in this chapter follows this four-stage structure, together with the keyword/-pattern recognition, a hand-traced dry run of the tabulation table being filled, and a time/space complexity comparison across all four stages.

1.2 Fibonacci Number

Problem Statement. Compute the n -th Fibonacci number, defined by $F(0) = 0$, $F(1) = 1$, and $F(n) = F(n-1) + F(n-2)$ for $n \geq 2$.

Pattern Recognition

“Each term is defined by combining a small, fixed number of earlier terms” is the simplest possible 1D DP keyword pattern, and Fibonacci is its archetype: the recurrence $F(n) = F(n-1) + F(n-2)$ is already explicitly given, so no derivation is needed – only the discipline of carrying it through all four stages described in Section 1.1.

Approach 1: Recursion

The recurrence translates directly into a function with two base cases ($n = 0$ and $n = 1$) and one recursive case.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int fibRecursive(int n) {
5     if (n <= 1) return n;
6     return fibRecursive(n - 1) + fibRecursive(n - 2);
7 }
8
9 int main() {
10     cout << fibRecursive(10) << endl; // 55
11     return 0;
12 }
```

Approach 1 Complexity

Time. Each call spawns two more, and the recursion tree has depth n , giving $O(2^n)$ – more precisely $O(\phi^n)$ where ϕ is the golden ratio, but exponential either way. **Space.** $O(n)$ for the recursion stack depth.

Approach 2: Memoization (Top-Down DP)

Why Fib(k) Is Computed Repeatedly

Expanding $\text{FIB}(5) = \text{FIB}(4) + \text{FIB}(3)$: the call to $\text{FIB}(4)$ itself expands to $\text{FIB}(3) + \text{FIB}(2)$, so $\text{FIB}(3)$ is computed once directly and once more inside $\text{FIB}(4)$'s expansion – and this doubling compounds at every level. Caching each $\text{FIB}(k)$'s result the first time it is computed, in an array *memo* indexed by k , means every later request for the same k is an $O(1)$ lookup instead of a full re-expansion.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int fibMemo(int n, vector<int>& memo) {
5     if (n <= 1) return n;
6     if (memo[n] != -1) return memo[n];
7
8     memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
9     return memo[n];
}
```

```

10 }
11
12 int main() {
13     int n = 10;
14     vector<int> memo(n + 1, -1);
15     cout << fibMemo(n, memo) << endl; // 55
16     return 0;
17 }

```

Approach 2 Complexity

Time. Each of the $n + 1$ distinct sub-problems $\text{FIB}(0), \dots, \text{FIB}(n)$ is computed exactly once, each doing $O(1)$ work beyond its (now-cached) recursive calls, giving $O(n)$. **Space.** $O(n)$ for the memo array, plus $O(n)$ for the recursion stack – still $O(n)$ overall, but for a different reason than Approach 1 (here it's the cache, not the redundant branching, that dominates the asymptotic class, even though the stack depth alone was already $O(n)$ in Approach 1 too).

Approach 3: Tabulation (Bottom-Up DP)

Filling the Table in the Opposite Direction

Memoization computes $\text{FIB}(n)$ by recursing *downward* toward the base cases, caching as it unwinds. Tabulation instead starts *at* the base cases and fills an array *dp upward* toward n in an explicit loop – computing the exact same values, in the same dependency order, but without any function call overhead.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int fibTabulation(int n) {
5     if (n == 0) return 0;
6
7     vector<int> dp(n + 1);
8     dp[0] = 0;
9     dp[1] = 1;
10    for (int i = 2; i <= n; i++) {
11        dp[i] = dp[i - 1] + dp[i - 2];
12    }
13    return dp[n];
14 }
15
16 int main() {
17     cout << fibTabulation(10) << endl; // 55
18     return 0;
19 }

```

Dry run of the tabulation table

Dry Run. Take $n = 6$.

i	0	1	2	3	4	5	6
$dp[i]$	0	1	1	2	3	5	8

Each entry from $i = 2$ onward is filled as $dp[i - 1] + dp[i - 2]$: e.g. $dp[4] = dp[3] + dp[2] = 2 + 1 = 3$. The answer, $dp[6] = 8$, matches $F(6) = 8$.

Approach 3 Complexity

Time. $O(n)$ – one pass filling $n - 1$ table entries. **Space.** $O(n)$ for the dp array; **no** recursion stack at all, unlike Approaches 1 and 2.

Approach 4: Space Optimization

Only the Last Two Entries Are Ever Needed

Computing $dp[i]$ only reads $dp[i - 1]$ and $dp[i - 2]$ – every entry before $i - 2$ is permanently dead the moment $dp[i]$ is computed. Replacing the full array with two rolling scalars, updated each iteration, preserves correctness while cutting space from $O(n)$ to $O(1)$.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int fibSpaceOptimized(int n) {
5     if (n == 0) return 0;
6
7     int prev2 = 0, prev1 = 1;
8     for (int i = 2; i <= n; i++) {
9         int current = prev1 + prev2;
10        prev2 = prev1;
11        prev1 = current;
12    }
13    return prev1;
14 }
15
16 int main() {
17     cout << fibSpaceOptimized(10) << endl; // 55
18     return 0;
19 }
```

Approach 4 Complexity

Time. $O(n)$, unchanged from tabulation. **Space.** $O(1)$ – only two scalar variables, regardless of how large n is.

Key Takeaway

Every later 1D DP problem in this chapter follows exactly this progression. The only thing that will change from here on is the *recurrence itself* – how many earlier states each step depends on, and how they are combined (summed, for counting; minimized or maximized, for optimization). The four-stage scaffolding shown here is reused verbatim.

1.3 Climbing Stairs

Problem Statement. A staircase has n steps. From any step, you may climb either 1 or 2 steps at a time. Count the number of distinct ways to reach the top (step n), starting from step 0.

Pattern Recognition

“Number of distinct ways to reach step n , taking 1 or 2 steps at a time” is the direct counting analogue of Section 1.2’s recurrence: the last move to reach step n was either a single step from $n - 1$, or a double step from $n - 2$ – so every way of reaching n is exactly one way of reaching $n - 1$, extended by one step, *or* one way of reaching $n - 2$, extended by two steps, and these two groups are disjoint and exhaustive.

Deriving the recurrence

Let $ways(i)$ be the number of distinct ways to reach step i . Since the only moves are $+1$ and $+2$, the last move into step i came from step $i - 1$ or step $i - 2$ – so $ways(i) = ways(i - 1) + ways(i - 2)$. Base cases: $ways(0) = 1$ (one way to be at the start: do nothing), and $ways(1) = 1$ (only one way to reach step 1: a single step from 0). This is exactly Fibonacci’s recurrence, shifted by one index, with different base cases.

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int climbStairsRecursive(int n) {
5     if (n <= 1) return 1;
6     return climbStairsRecursive(n - 1) + climbStairsRecursive(n - 2);
7 }
8
9 int main() {
10     cout << climbStairsRecursive(5) << endl; // 8
11     return 0;
12 }
```

Approach 1 Complexity

Time. $O(2^n)$ – identical overlapping-subproblem blowup as plain Fibonacci. **Space.** $O(n)$ for the recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int climbStairsMemo(int n, vector<int>& memo) {
5     if (n <= 1) return 1;
6     if (memo[n] != -1) return memo[n];
7
8     memo[n] = climbStairsMemo(n - 1, memo) + climbStairsMemo(n - 2,
9         memo);
10    return memo[n];
11 }
12
13 int main() {
14     int n = 5;
15     vector<int> memo(n + 1, -1);
16     cout << climbStairsMemo(n, memo) << endl; // 8
17 }
```

```

16     return 0;
17 }

```

Approach 2 Complexity

Time. $O(n)$. **Space.** $O(n)$ memo array + $O(n)$ recursion stack.

Approach 3: Tabulation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int climbStairsTabulation(int n) {
5      if (n <= 1) return 1;
6
7      vector<int> dp(n + 1);
8      dp[0] = 1;
9      dp[1] = 1;
10     for (int i = 2; i <= n; i++) {
11         dp[i] = dp[i - 1] + dp[i - 2];
12     }
13     return dp[n];
14 }
15
16 int main() {
17     cout << climbStairsTabulation(5) << endl; // 8
18     return 0;
19 }

```

Dry run of the tabulation table

Dry Run. Take $n = 5$.

i	0	1	2	3	4	5
$dp[i]$	1	1	2	3	5	8

$dp[5] = 8$: indeed, there are 8 distinct ways to climb 5 steps using moves of size 1 or 2 (e.g. 1+1+1+1+1, 2+1+1+1 in every position, 2+2+1 in every order, and so on).

Approach 3 Complexity

Time. $O(n)$. **Space.** $O(n)$ for the table, no recursion stack.

Approach 4: Space Optimization

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int climbStairsSpaceOptimized(int n) {
5      if (n <= 1) return 1;
6
7      int prev2 = 1, prev1 = 1;
8      for (int i = 2; i <= n; i++) {
9          int current = prev1 + prev2;

```

```

10     prev2 = prev1;
11     prev1 = current;
12 }
13 return prev1;
14 }
15
16 int main() {
17     cout << climbStairsSpaceOptimized(5) << endl; // 8
18     return 0;
19 }

```

Approach 4 Complexity

Time. $O(n)$. **Space.** $O(1)$.

Key Takeaway

Recognizing that “count the ways to reach n via moves from a small fixed set” always produces a recurrence that **sums** the way-counts of the positions those moves originate from is the key insight – the specific move set (here, $\{1, 2\}$) only determines *how many* earlier terms the recurrence sums, and which ones. Section 1.4 generalizes this from counting ways to minimizing cost, with the sum replaced by a minimum.

1.4 Frog Jump

Problem Statement. A frog starts at index 0 of an array *height* and wants to reach index $n - 1$. From index i , it may jump to $i + 1$ or $i + 2$, at a cost of $|height[i] - height[j]|$ for a jump to index j . Find the minimum total cost to reach index $n - 1$.

Pattern Recognition

“Minimum cost to reach position n , moving via a small fixed set of jump sizes” is Section 1.3’s recurrence shape with **sum replaced by minimum**: instead of *counting* every way to combine smaller answers, we *minimize* over them, because here we want the cheapest path, not a tally of all paths.

Deriving the recurrence

Let $minCost(i)$ be the minimum cost to reach index i from index 0. The frog’s last jump into index i came from either $i - 1$ (cost $|height[i] - height[i - 1]|$ added to $minCost(i - 1)$) or $i - 2$ (cost $|height[i] - height[i - 2]|$ added to $minCost(i - 2)$) – and since we want the *cheapest* way to reach i , we take the minimum of these two options:

$$minCost(i) = \min (minCost(i-1) + |height[i] - height[i-1]|, minCost(i-2) + |height[i] - height[i-2]|).$$

Base case: $minCost(0) = 0$ (no cost to start where you already are); for $i = 1$, only the jump from 0 is possible (there is no index -1), so $minCost(1) = |height[1] - height[0]|$ directly.

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int frogJumpRecursive(int i, vector<int>& height) {

```

```

5     if (i == 0) return 0;
6     if (i == 1) return abs(height[1] - height[0]);
7
8     int fromOneBack = frogJumpRecursive(i - 1, height) + abs(height[i]
9         - height[i-1]);
10    int fromTwoBack = frogJumpRecursive(i - 2, height) + abs(height[i]
11        - height[i-2]);
12    return min(fromOneBack, fromTwoBack);
13 }
14
15 int main() {
16     vector<int> height = {10, 30, 40, 50};
17     int n = height.size();
18     cout << frogJumpRecursive(n - 1, height) << endl; // 40
19     return 0;
20 }

```

Approach 1 Complexity

Time. $O(2^n)$ – the same two-branches-per-call structure as every recurrence in this chapter so far. **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int frogJumpMemo(int i, vector<int>& height, vector<int>& memo) {
5      if (i == 0) return 0;
6      if (i == 1) return abs(height[1] - height[0]);
7      if (memo[i] != -1) return memo[i];
8
9      int fromOneBack = frogJumpMemo(i - 1, height, memo) + abs(height[i]
10         - height[i-1]);
11     int fromTwoBack = frogJumpMemo(i - 2, height, memo) + abs(height[i]
12         - height[i-2]);
13     memo[i] = min(fromOneBack, fromTwoBack);
14     return memo[i];
15 }
16
17 int main() {
18     vector<int> height = {10, 30, 40, 50};
19     int n = height.size();
20     vector<int> memo(n, -1);
21     cout << frogJumpMemo(n - 1, height, memo) << endl; // 40
22     return 0;
23 }

```

Approach 2 Complexity

Time. $O(n)$. **Space.** $O(n)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int frogJumpTabulation(vector<int>& height) {
5     int n = height.size();
6     vector<int> dp(n);
7     dp[0] = 0;
8     if (n > 1) dp[1] = abs(height[1] - height[0]);
9
10    for (int i = 2; i < n; i++) {
11        int fromOneBack = dp[i - 1] + abs(height[i] - height[i-1]);
12        int fromTwoBack = dp[i - 2] + abs(height[i] - height[i-2]);
13        dp[i] = min(fromOneBack, fromTwoBack);
14    }
15    return dp[n - 1];
16 }
17
18 int main() {
19     vector<int> height = {10, 30, 40, 50};
20     cout << frogJumpTabulation(height) << endl; // 40
21     return 0;
22 }

```

Dry run of the tabulation table

Dry Run. Take $height = [10, 30, 40, 50]$.

- $dp[0] = 0$.
- $dp[1] = |30 - 10| = 20$.
- $dp[2] = \min(dp[1] + |40 - 30|, dp[0] + |40 - 10|) = \min(20 + 10, 0 + 30) = \min(30, 30) = 30$.
- $dp[3] = \min(dp[2] + |50 - 40|, dp[1] + |50 - 30|) = \min(30 + 10, 20 + 20) = \min(40, 40) = 40$.

i	0	1	2	3
$height[i]$	10	30	40	50
$dp[i]$	0	20	30	40

The answer is $dp[3] = 40$: one cheapest path is $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$, costing $20 + 10 + 10 = 40$ (and the alternative $0 \rightarrow 2 \rightarrow 3$, costing $30 + 10 = 40$, ties exactly).

Approach 3 Complexity

Time. $O(n)$. **Space.** $O(n)$ table, no recursion stack.

Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int frogJumpSpaceOptimized(vector<int>& height) {
5     int n = height.size();
6     if (n == 1) return 0;
7

```

```

8     int prev2 = 0, prev1 = abs(height[1] - height[0]);
9     for (int i = 2; i < n; i++) {
10        int fromOneBack = prev1 + abs(height[i] - height[i-1]);
11        int fromTwoBack = prev2 + abs(height[i] - height[i-2]);
12        int current = min(fromOneBack, fromTwoBack);
13        prev2 = prev1;
14        prev1 = current;
15    }
16    return prev1;
17 }
18
19 int main() {
20     vector<int> height = {10, 30, 40, 50};
21     cout << frogJumpSpaceOptimized(height) << endl; // 40
22     return 0;
23 }

```

Approach 4 Complexity

Time. $O(n)$. **Space.** $O(1)$.

Key Takeaway

Swapping a sum for a minimum (or maximum) in the combination step is often the *only* change needed to turn a “counting” DP recurrence into a “cost optimization” one, with every other part of the four-stage derivation – base cases, indexing, the eventual space optimization – carrying over in the same shape. Section 1.5 generalizes the jump set from $\{1, 2\}$ to $\{1, \dots, k\}$, widening the minimum from two candidates to k .

1.5 Frog Jump with K Distances

Problem Statement. Same setting as Section 1.4, except the frog may jump from index i to any of $i+1, i+2, \dots, i+k$ (up to k steps ahead, instead of only 1 or 2), at cost $|height[i] - height[j]|$ for a jump to index j . Find the minimum total cost to reach index $n - 1$.

Pattern Recognition

“Same recurrence shape, but the fixed set of options widens from 2 to a parameter k ” – recognizing this is the entire content of this section: Section 1.4’s minimum over 2 candidates ($i - 1$ and $i - 2$) becomes a minimum over up to k candidates ($i - 1, i - 2, \dots, i - k$), with everything else about the derivation unchanged.

Deriving the recurrence

The frog’s last jump into index i came from some index $i - j$ for some j between 1 and k (as long as $i - j \geq 0$). So:

$$\text{minCost}(i) = \min_{1 \leq j \leq k, i-j \geq 0} \left(\text{minCost}(i - j) + |height[i] - height[i - j]| \right).$$

Base case: $\text{minCost}(0) = 0$.

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int frogJumpKRecursive(int i, int k, vector<int>& height) {
5     if (i == 0) return 0;
6
7     int best = INT_MAX;
8     for (int j = 1; j <= k && i - j >= 0; j++) {
9         int cost = frogJumpKRecursive(i - j, k, height) +
10                 abs(height[i] - height[i - j]);
11         best = min(best, cost);
12     }
13     return best;
14 }
15
16 int main() {
17     vector<int> height = {10, 30, 40, 50, 20};
18     int n = height.size(), k = 3;
19     cout << frogJumpKRecursive(n - 1, k, height) << endl;
20     return 0;
21 }

```

Approach 1 Complexity

Time. $O(k^n)$ in the worst case – each call now branches into up to k recursive calls instead of 2. **Space.** $O(n)$ for the recursion stack depth.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int frogJumpKMemo(int i, int k, vector<int>& height, vector<int>& memo)
5 {
6     if (i == 0) return 0;
7     if (memo[i] != -1) return memo[i];
8
9     int best = INT_MAX;
10    for (int j = 1; j <= k && i - j >= 0; j++) {
11        int cost = frogJumpKMemo(i - j, k, height, memo) +
12                abs(height[i] - height[i - j]);
13        best = min(best, cost);
14    }
15    return memo[i] = best;
16 }
17
18 int main() {
19     vector<int> height = {10, 30, 40, 50, 20};
20     int n = height.size(), k = 3;
21     vector<int> memo(n, -1);
22     cout << frogJumpKMemo(n - 1, k, height, memo) << endl;
23     return 0;
24 }

```

Approach 2 Complexity

Time. $O(n \cdot k)$ – n distinct sub-problems, each doing $O(k)$ work in its loop. **Space.** $O(n)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int frogJumpKTabulation(vector<int>& height, int k) {
5     int n = height.size();
6     vector<int> dp(n, 0);
7
8     for (int i = 1; i < n; i++) {
9         int best = INT_MAX;
10        for (int j = 1; j <= k && i - j >= 0; j++) {
11            int cost = dp[i - j] + abs(height[i] - height[i - j]);
12            best = min(best, cost);
13        }
14        dp[i] = best;
15    }
16    return dp[n - 1];
17 }
18
19 int main() {
20     vector<int> height = {10, 30, 40, 50, 20};
21     cout << frogJumpKTabulation(height, 3) << endl;
22     return 0;
23 }

```

Dry run of the tabulation table

Dry Run. Take $height = [10, 30, 40, 50, 20]$, $k = 3$.

- $dp[0] = 0$.
- $dp[1] = dp[0] + |30-10| = 20$ (only $j = 1$ possible).
- $dp[2] = \min(dp[1] + |40-30|, dp[0] + |40-10|) = \min(30, 30) = 30$.
- $dp[3] = \min(dp[2] + |50-40|, dp[1] + |50-30|, dp[0] + |50-10|) = \min(40, 40, 40) = 40$.
- $dp[4] = \min(dp[3] + |20-50|, dp[2] + |20-40|, dp[1] + |20-30|) = \min(40+30, 30+20, 20+10) = \min(70, 50, 30) = 30$.

i	0	1	2	3	4
$dp[i]$	0	20	30	40	30

The answer is $dp[4] = 30$, achieved by jumping directly from index 1 ($height = 30$) to index 4 ($height = 20$), a jump of size $3 \leq k$, costing $dp[1] + |20 - 30| = 20 + 10 = 30$.

Approach 3 Complexity

Time. $O(n \cdot k)$. **Space.** $O(n)$ table, no recursion stack.

Approach 4: Space Optimization

Why This One Cannot Be Reduced All the Way to $O(1)$

Unlike Section 1.4, where only the *two* immediately preceding entries were ever needed, here computing $dp[i]$ may need any of the previous k entries – so the space optimization here reduces storage to a **sliding window** of the last k entries (e.g. a small circular buffer or deque of size k), rather than collapsing all the way to two scalar variables. When k is a fixed small constant, this is still $O(k) = O(1)$ relative to n ; when k can grow proportionally with n , no reduction below $O(n)$ is possible, since up to n of the entries might genuinely be needed.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int frogJumpKSpaceOptimized(vector<int>& height, int k) {
5     int n = height.size();
6     vector<int> window(k, 0); // window[i % k] holds dp[i]
7     window[0] = 0;
8
9     for (int i = 1; i < n; i++) {
10         int best = INT_MAX;
11         for (int j = 1; j <= k && i - j >= 0; j++) {
12             int cost = window[(i - j) % k] + abs(height[i] - height[i -
13                 j]);
14             best = min(best, cost);
15         }
16         window[i % k] = best;
17     }
18     return window[(n - 1) % k];
19 }
20
21 int main() {
22     vector<int> height = {10, 30, 40, 50, 20};
23     cout << frogJumpKSpaceOptimized(height, 3) << endl;
24     return 0;
25 }
```

Approach 4 Complexity

Time. $O(n \cdot k)$, unchanged. **Space.** $O(k)$ for the sliding window, instead of $O(n)$ for the full table.

Key Takeaway

Generalizing a recurrence’s fixed-size option set from a small constant (here, 2, in Frog Jump) to a parameter k widens every stage’s loop from 2 iterations to k , multiplying the time complexity by k and capping how far the space optimization can shrink: only the last k table entries, not just the last 2, must remain available, so the “two rolling variables” trick of earlier sections becomes a “sliding window of size k ” trick here.

1.6 House Robber

Problem Statement. Given an array *nums* where *nums*[*i*] is the amount of money in house *i*, find the maximum total amount that can be robbed without ever robbing two **adjacent** houses.

Pattern Recognition

“Maximum sum, no two adjacent elements chosen” is a different recurrence *shape* from every problem so far in this chapter: those were all “combine answers from a fixed set of earlier positions” (summed, for counting, or minimized, for cost). This one is a genuine **include-or-exclude choice** at every position – structurally the same choice as the pick/not-pick recursion from the Recursion chapter’s Power Set section, but now resolved by **maximizing** rather than branching into both options forever.

Deriving the recurrence

Let $f(i)$ be the maximum amount robbable considering only houses 0 through i . At house i , there are exactly two choices: **rob** house i (gaining $nums[i]$, but then house $i - 1$ *must* be skipped, so we add the best answer from $f(i - 2)$), or **skip** house i entirely (carrying forward $f(i - 1)$ unchanged, since skipping loses nothing already secured). We take whichever choice is larger:

$$f(i) = \max(f(i - 1), f(i - 2) + nums[i]).$$

Base cases: $f(0) = nums[0]$ (only one house, must take it for the best possible answer at this point), and, treating an empty prefix before index 0 as contributing 0, we can write $f(-1) = 0$ for notational convenience when computing $f(1) = \max(f(0), f(-1) + nums[1]) = \max(nums[0], nums[1])$.

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int robRecursive(int i, vector<int>& nums) {
5     if (i == 0) return nums[0];
6     if (i < 0) return 0;
7
8     int skip = robRecursive(i - 1, nums);
9     int take = robRecursive(i - 2, nums) + nums[i];
10    return max(skip, take);
11 }
12
13 int main() {
14     vector<int> nums = {1, 2, 3, 1};
15     int n = nums.size();
16     cout << robRecursive(n - 1, nums) << endl; // 4
17     return 0;
18 }
```

Approach 1 Complexity

Time. $O(2^n)$. **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int robMemo(int i, vector<int>& nums, vector<int>& memo) {
```

```

5     if (i == 0) return nums[0];
6     if (i < 0) return 0;
7     if (memo[i] != -1) return memo[i];
8
9     int skip = robMemo(i - 1, nums, memo);
10    int take = robMemo(i - 2, nums, memo) + nums[i];
11    return memo[i] = max(skip, take);
12 }
13
14 int main() {
15     vector<int> nums = {1, 2, 3, 1};
16     int n = nums.size();
17     vector<int> memo(n, -1);
18     cout << robMemo(n - 1, nums, memo) << endl; // 4
19     return 0;
20 }

```

Approach 2 Complexity

Time. $O(n)$. **Space.** $O(n)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int robTabulation(vector<int>& nums) {
5     int n = nums.size();
6     vector<int> dp(n);
7     dp[0] = nums[0];
8
9     for (int i = 1; i < n; i++) {
10        int skip = dp[i - 1];
11        int take = (i >= 2 ? dp[i - 2] : 0) + nums[i];
12        dp[i] = max(skip, take);
13    }
14    return dp[n - 1];
15 }
16
17 int main() {
18     vector<int> nums = {1, 2, 3, 1};
19     cout << robTabulation(nums) << endl; // 4
20     return 0;
21 }

```

Dry run of the tabulation table

Dry Run. Take $nums = [1, 2, 3, 1]$.

- $dp[0] = 1$.
- $dp[1] = \max(dp[0], 0 + 2) = \max(1, 2) = 2$.
- $dp[2] = \max(dp[1], dp[0] + 3) = \max(2, 1 + 3) = \max(2, 4) = 4$.
- $dp[3] = \max(dp[2], dp[1] + 1) = \max(4, 2 + 1) = \max(4, 3) = 4$.

i	0	1	2	3
$nums[i]$	1	2	3	1
$dp[i]$	1	2	4	4

The answer is $dp[3] = 4$, achieved by robbing houses 0 and 2 (values $1 + 3 = 4$), which beats robbing house 1 alone or house 3 alone or houses 1, 3 together ($2 + 1 = 3$).

Approach 3 Complexity

Time. $O(n)$. **Space.** $O(n)$ table, no recursion stack.

Approach 4: Space Optimization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int robSpaceOptimized(vector<int>& nums) {
5     int n = nums.size();
6     int prev2 = 0, prev1 = nums[0];
7
8     for (int i = 1; i < n; i++) {
9         int skip = prev1;
10        int take = prev2 + nums[i];
11        int current = max(skip, take);
12        prev2 = prev1;
13        prev1 = current;
14    }
15    return prev1;
16 }
17
18 int main() {
19     vector<int> nums = {1, 2, 3, 1};
20     cout << robSpaceOptimized(nums) << endl; // 4
21     return 0;
22 }
```

Approach 4 Complexity

Time. $O(n)$. **Space.** $O(1)$.

Key Takeaway

“Maximum sum subject to a no-two-adjacent constraint” is solved by an include-or-exclude recurrence at every position, maximized rather than summed – the DP analogue of the pick/not-pick backtracking tree from the Recursion chapter’s Power Set section, but resolved in $O(n)$ instead of $O(2^n)$ because we only need the *best* outcome at each position, not every possible outcome. Section 1.7 adapts this exact recurrence to a circular arrangement of houses.

1.7 House Robber II (Houses in a Circle)

Problem Statement. Same as Section 1.6, except the houses are arranged in a **circle** – house 0 and house $n - 1$ are now also considered adjacent, so robbing both of them together is forbidden, in addition to the usual no-two-consecutive-houses rule.

Pattern Recognition

“Same problem, but the arrangement is circular instead of linear” is a keyword for a **reduction** technique: rather than deriving a new recurrence that somehow accounts for a wraparound adjacency, we instead run the *already-solved* linear House Robber (Section 1.6) twice, on two carefully chosen linear sub-ranges, and combine their results.

Understanding the problem carefully

The only new constraint, compared to the linear version, is that house 0 and house $n - 1$ cannot *both* be robbed. Equivalently: in any valid circular solution, **at least one** of house 0 or house $n - 1$ is never robbed. This means every valid circular solution is also a valid *linear* solution either to the sub-array excluding house 0 (i.e. houses 1 through $n - 1$), or to the sub-array excluding house $n - 1$ (i.e. houses 0 through $n - 2$) – because whichever of the two “forbidden together” houses is excluded from the solution can simply be excluded from consideration entirely.

Reduce to Two Linear Sub-problems, Take the Better One

Run Section 1.6’s linear House Robber on the sub-array `nums[0..n - 2]` (excluding the last house), and again on `nums[1..n - 1]` (excluding the first house). The answer to the circular version is the **maximum** of these two results.

Why considering exactly these two sub-ranges is sufficient

Every valid circular solution falls into (at least) one of two categories: those that do not rob house $n - 1$ (entirely captured by the linear sub-problem on houses 0 through $n - 2$, since within that range the usual linear adjacency rule is the *only* remaining constraint), and those that do not rob house 0 (captured by the linear sub-problem on houses 1 through $n - 1$). A solution that robs *neither* house 0 nor house $n - 1$ is captured by both sub-problems redundantly, which costs nothing – we just take whichever of the two computed maximums is larger, and that larger value is guaranteed to be a valid circular answer that misses no better possibility, since any valid circular solution at all must belong to at least one of these two categories.

Approach 1: Recursion (via Two Calls to the Linear Helper)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int robRecursive(int i, vector<int>& nums) {
5     if (i == 0) return nums[0];
6     if (i < 0) return 0;
7
8     int skip = robRecursive(i - 1, nums);
9     int take = robRecursive(i - 2, nums) + nums[i];
10    return max(skip, take);
11 }
12
13 int robLinear(vector<int> sub) {
14     if (sub.empty()) return 0;
15     return robRecursive((int)sub.size() - 1, sub);
16 }
17
18 int robCircular(vector<int>& nums) {
19     int n = nums.size();

```

```

20     if (n == 1) return nums[0];
21
22     vector<int> excludeLast(nums.begin(), nums.end() - 1);
23     vector<int> excludeFirst(nums.begin() + 1, nums.end());
24
25     return max(robLinear(excludeLast), robLinear(excludeFirst));
26 }
27
28 int main() {
29     vector<int> nums = {2, 3, 2};
30     cout << robCircular(nums) << endl; // 3
31     return 0;
32 }

```

Approach 1 Complexity

Time. $O(2^n)$ for each of the two linear calls, so $O(2^n)$ overall (the constant factor of 2 calls does not change the asymptotic class). **Space.** $O(n)$ recursion stack.

Approaches 2–4: Memoization, Tabulation, and Space Optimization

Each of these three later stages is obtained by substituting Section 1.6's corresponding memoized, tabulated, or space-optimized linear House Robber implementation in place of *robLinear* above – no new derivation is required, since the reduction itself (call the linear solver twice, take the max) is independent of which of the four stages that linear solver internally uses.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int robLinearSpaceOptimized(vector<int>& nums) {
5      if (nums.empty()) return 0;
6      int n = nums.size();
7      int prev2 = 0, prev1 = nums[0];
8
9      for (int i = 1; i < n; i++) {
10         int skip = prev1;
11         int take = prev2 + nums[i];
12         int current = max(skip, take);
13         prev2 = prev1;
14         prev1 = current;
15     }
16     return prev1;
17 }
18
19 int robCircularSpaceOptimized(vector<int>& nums) {
20     int n = nums.size();
21     if (n == 1) return nums[0];
22
23     vector<int> excludeLast(nums.begin(), nums.end() - 1);
24     vector<int> excludeFirst(nums.begin() + 1, nums.end());
25
26     return max(robLinearSpaceOptimized(excludeLast),
27               robLinearSpaceOptimized(excludeFirst));
28 }
29
30 int main() {
31     vector<int> nums = {2, 3, 2};

```

```
32     cout << robCircularSpaceOptimized(nums) << endl; // 3
33     return 0;
34 }
```

Dry run

Dry Run. Take $nums = [2, 3, 2]$.

- Exclude last (sub-array $[2, 3]$): linear House Robber gives $\max(2, 3) = 3$.
- Exclude first (sub-array $[3, 2]$): linear House Robber gives $\max(3, 2) = 3$.
- Overall answer: $\max(3, 3) = 3$.

Checking by hand: robbing houses 0 and 2 together is forbidden (circular adjacency), so the best options are house 1 alone (3), or house 0 alone (2), or house 2 alone (2) – confirming 3 is correct.

Final (Space-Optimized) Complexity

Time. $O(n)$ – two linear passes, each $O(n)$. **Space.** $O(1)$ extra (beyond the two $O(n)$ sliced sub-arrays, which could themselves be avoided by passing index ranges instead of copying, reducing auxiliary space to true $O(1)$).

Key Takeaway

When a new constraint (here, circularity) only affects the boundary between two elements, check whether the problem can be split into a small, fixed number of cases that each remove the offending boundary entirely – reducing the new problem to multiple calls of an already-solved simpler one, rather than re-deriving a new recurrence from scratch. This reduction technique is often faster to get correct than attempting to encode the wraparound constraint directly into a single, more complex recurrence.